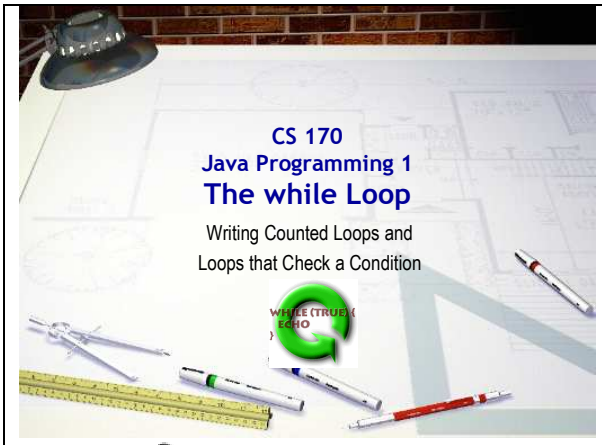


Slide 1

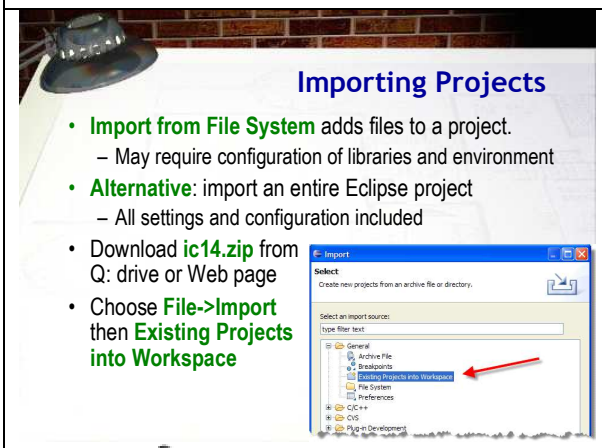
CS 170
Java Programming 1
The while Loop
Duration: 00:00:60
Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture on the while loop. The while loop is a more general purpose loop than the for loop, which is really optimized for writing counted loops. You can also write counted loops using the while loop, and since you've already learned how to do that using for, that's where we'll start.

The while loop really comes into its own, though, when it comes time to write more complex loops that don't use a simple counter for their bounds. These loops, called indefinite loops, check a condition or combination of conditions before deciding whether to continue or not.

Before we get to loops, though, let's learn how to import an entire project into Eclipse. If you're working at home, you'll want to complete the optional Eclipse@Home lecture first to make sure that you have Eclipse set up correctly for your home machine. If you're working in the OCC Computing Center, you won't need to set up Eclipse.



Slide 2

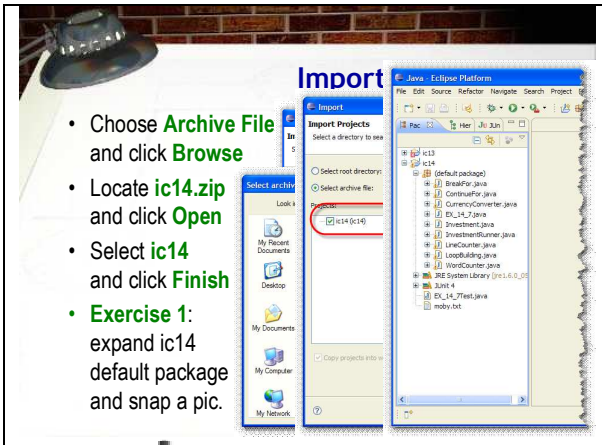
Importing Projects
Duration: 00:01:13
Advance mode: Auto

Notes:

In lecture this week, you learned how to create an Eclipse project and then to import individual files into that project using the Import from File System function. When you do that, though, any libraries you use, such as the JUnit testing library you used in EX_13_7, need to be reconfigured for that individual project.

An alternative to this file-by-file import is to import an entire Eclipse project. When you do that, all of the customized settings and configuration are imported along with the files. That's what we'll do for the in-class exercise project for the online lectures this week. Here's what you should do.

- First, download the file ic14.zip from the Q: drive (if working in the lab), or from the class Web page, and store it on your local machine. (Don't put it inside your workspace folder, though).
- From the File menu, choose Import and, on the Import dialog shown here, choose Existing Projects into Workspace and click the Next button.



Slide 3

Importing Projects

Duration: 00:01:44
Advance mode: Auto

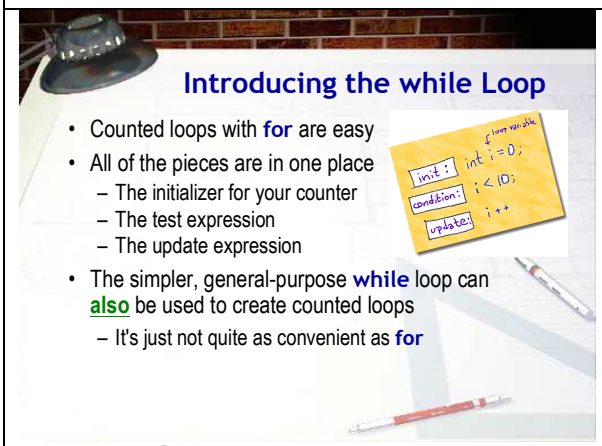
Notes:

When the next page in the Import Wizard appears you'll be given a chance to import a project from a directory on the file system or from an archive file. Use the file system option if you want to copy a project that is contained in another Eclipse workspace. You want to open the project contained inside the ic14.zip file that you downloaded, though, so click the radio button that says "Select Archive File" and then click the Browse button.

When the file-choose dialog appears, its title will tell you to "Select the archive containing the projects to import". In our case that means the ic14.zip file that you just downloaded. Once you've located it, click the Open button to import it.

When the Import dialog reappears, you'll see a list of projects in the center Window. If the archive file contains multiple projects, you can select only the ones you want to import. One caution, though. You can't import a project if your workspace already has a project with the same name. In that case, you need to rename the existing project (using Refactor->Rename on the context menu) before importing the new project. In our case, there's only one project, ic14. Make sure that it's selected and click the Finish button to import into your workspace.

Now let's get started on our IC exercise document for the online portion of the class this week. Once you have it set up, for Exercise 1, expand the default package for the ic14 project and shoot me a screen-shot so that I can see you were able to import the project correctly.



Slide 4

Introducing the while Loop

Duration: 00:00:44
Advance mode: Auto

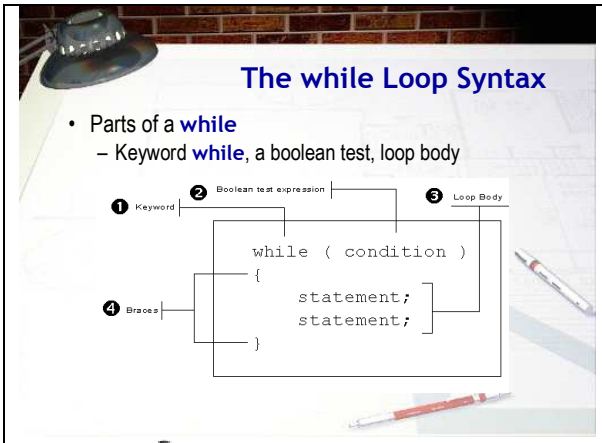
Notes:

Ok, with that out of the way, let's turn our attention to the while loop.

The for loop makes writing counted loops very easy because it puts the pieces you need—a place to initialize your counter, a place to test your counter, and a place to update your counter—all within easy reach. If you forget to initialize your counter, that empty space in the for-loop condition really stands out.

You can, however, do exactly the same thing with Java's simpler, but more general-purpose while loop; to my mind, though, using a while loop is just not quite as convenient.

Let's write a few, though, and you can make up your own mind.



Slide 5

The while Loop Syntax

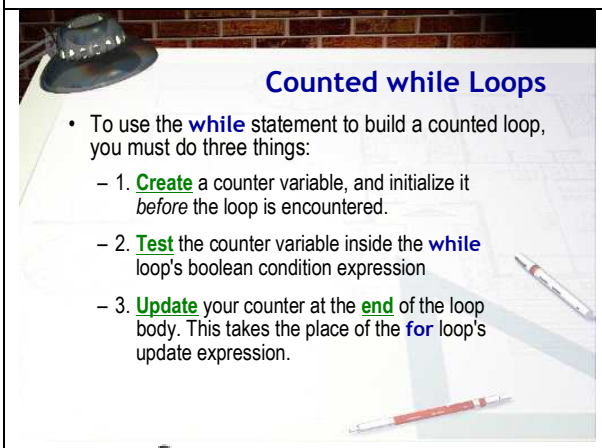
Duration: 00:00:49
Advance mode: Auto

Notes:

Let's start by looking at the while loop syntax. The while loop only has three parts:

- The keyword while (instead of for or if)
- A boolean condition. This acts exactly like the boolean condition inside an if statement or the middle, test condition, inside a for loop header.
- A loop body where the statements are placed. If you have multiple statements inside your loop body, you must have braces around them. If the loop body consists of only a single statement, then braces aren't required but I'd still recommend them. Always adding braces to every loop body will help you avoid some common bugs in your code.

Note that there are no initialization expressions or update expressions like there are with a for loop.



Slide 6

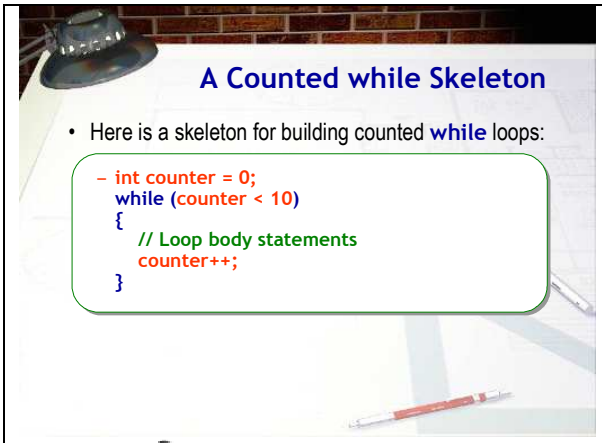
Counted while Loops

Duration: 00:01:16
Advance mode: Auto

Notes:

Now, since the while loop doesn't come with a built-in place to initialize or update a counter, if you want to build a counter-controlled loop using the while statement, you'll need to manually do three things:

- First, you'll need to create a counter variable, and initialize it before the loop starts. This takes the place of the initialization expression in the for loop. Make sure that you create your counter before the loop condition, not at the beginning of the loop body. If you create your counter variable inside the loop body, it will get recreated and reinitialized every time your loop repeats.
- Second, you'll need to test the counter variable inside the while condition against the value that represents the loop's upper bounds or ending point. This works just like the test, or middle expression in the for loop.
- Finally, at the end of the loop body you'll need to update your loop counter. This takes the place of the update expression in the for loop. Make sure that this update expression follows all of the action statements inside the loop body, if you want your loop to act like the equivalent for loop.



Slide 7

A Counted while Skeleton

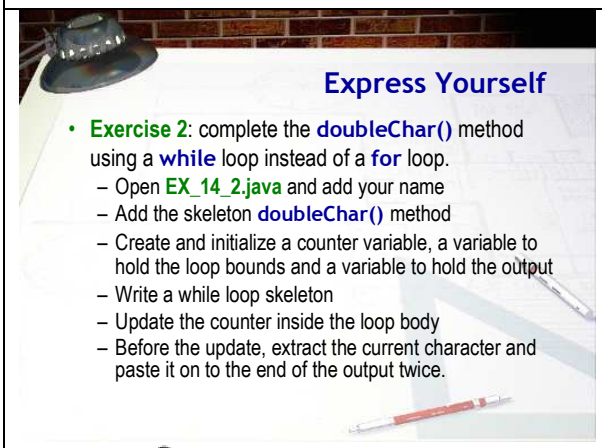
Duration: 00:00:39
Advance mode: Auto

Notes:

Here's the basic skeleton for a counted loop, written with while. Notice:

- the counter variable, declared and initialized before the loop starts.
- the loop bounds condition that uses tests the counter against the loop bounds. In this case the upper bounds is the literal value 10.
- the update expression that changes the value of the counter and is placed at the bottom of the loop body, after all of the normal loop actions are finished.

You can use this skeleton as a kind of "checklist" every time you need to write a counter-controlled loop using the while statement.



Slide 8

Express Yourself

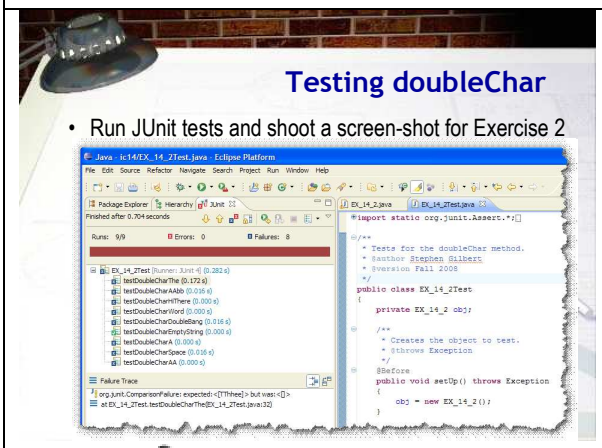
Duration: 00:02:04
Advance mode: Auto

Notes:

Let's see if you can put this to work on a program that you've already seen. In class this week you completed the doubleChar() method in EX_13_7.java. Let's complete the same method using a while loop. Here are the steps to follow.

- First, open EX_14_2.java and add your name to the comment.
- Next, write the method skeleton. The doubleChar method returns a String and take a single String parameter. Since it isn't a void method, you'll need to add a return statement. To get the skeleton to work, just return the empty String. Make sure that your code compiles at this point.
- Create a counter variable and initialize it to 0. You'll use this counter to control the loop.
- Create a second variable to store the upper-bounds of your loop. Since doubleChar is going to go through the loop once for every character in the input string, this should be set to the length of the input String. You can find that by calling the String's length() method.
- Create a third variable to hold the method's output. Since the output will be a String, you can initialize it to the empty String.
- Add the while loop skeleton. Your loop should continue while your counter is less than the bounds.
- At the end of the loop body, you need to update your counter variable.
- Inside the loop body, before the update expression, add the statements to extract the current character (you'll use your counter and the charAt() method for that), and then

paste that character twice on to the end of the output String. Use concatenation for that.



Slide 9

Testing doubleChar

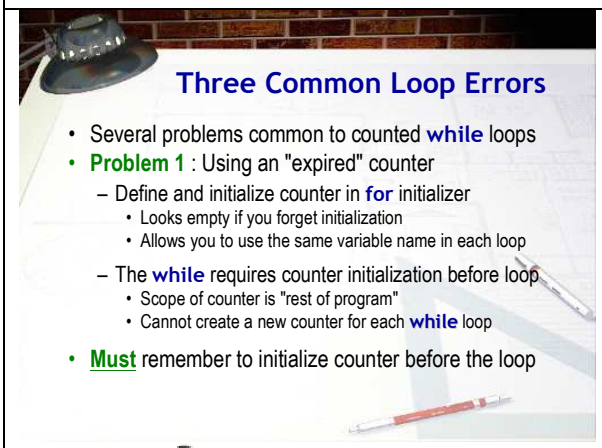
Duration: 00:01:35

Advance mode: Auto

Notes:

To test if you've completed Exercise 2 correctly, I want you to run the same set of JUnit tests we ran in class. In this project, those are in the file EX_14_2Test.java. Because the test file caused an error when included in the project (since you hadn't yet written the doubleChar() method), the ic14 project excludes the file from the Java build path. To run the JUnit tests, you'll first need to include it. Here's how:

- Locate the file EX_14_2Test.java at the very bottom of the project. Notice that it is NOT inside the default package with the other Java classes and that its icon looks a little different.
- Right click the file and from the context menu select Build Path and then Include. This will move the file into the default package along with the other Java source files.
- Finally, open up the file in your editor and then run it as a JUnit test suite. In my example here, you can see that my version still fails almost all of the tests. I want you to work on the method until all of the tests pass, and then shoot a screen-shot and paste it into your in-class exercise document as Exercise 2. Underneath your JUnit screen-shot, copy and past the actual source code for the doubleChar() method, so I can see your while-loop code.



Slide 10

Three Common Loop Errors

Duration: 00:02:11

Advance mode: Auto

Notes:

There are three common errors that often occur when writing counted loops using while. By learning what they are, and then checking for each one when you write your loops, you increase your chances of writing a correct loop the first time.

The first problem occurs when you use an "expired" counter. When using the for loop, you usually create and initialize your counter variables inside the initialization expression of the for loop. If you forget to do so, the for loop condition usually looks "empty", which acts as a reminder.

With the while loop, however, you must define and initialize your counter outside the while loop. This has several implications:

- In the for loop, the scope of the counter variable is restricted to the body of the loop. That is not true for the

while loop.

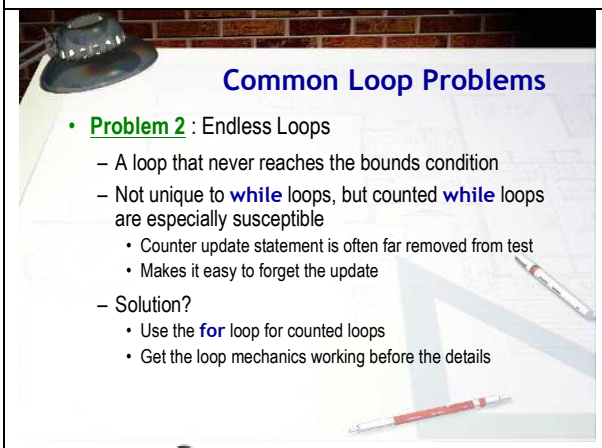
- If you have several while loops in the same method, and each of them use the same counter variable name, then you cannot re-declare the variable for each new loop. All of your loops use the same variable.

You must, however, remember to reinitialize the counter before each loop, even though you can't redefine it. Forgetting to do this often leads to the case of the expired counter. If a counted while loop is preceded by another, and both use the same counter variable, you may forget to reinitialize the counter for the second loop. This doesn't create a syntax error, but it does give you the wrong answer.

There are two possible solutions to the expired counter problem.

If you give all of your counters unique names, you won't run into the problem. Sometimes, however, this is hard to do. Thinking up a new name when you really want to use the common counter names like i, idx, or counter, can be difficult.

A better solution is to train yourself to always initialize your counter on the line immediately preceding the while condition. After a bit, a missing initialization starts to look as "empty" as the missing initialization section in a for loop.



Slide 11

Common Loop Problems

Duration: 00:01:27

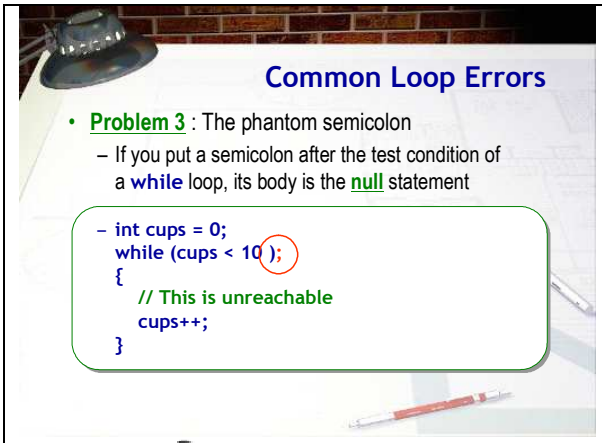
Advance mode: Auto

Notes:

The second common loop problem is the "endless" loop. Endless loops are also called infinite or endless loops. The problem of endless loops is not unique to counted while loops. You can create endless loops in a wide variety of ways. Still, a counted while loop lends itself to endless loops in a way that the others don't.

Because the while loop places the counter update expression at the bottom of the loop—which may be far removed from the while loop condition by the time all of the intervening statements have been added—it is easy to forget it. With the for loop, leaving out the update expression creates an "empty" looking loop condition, but not with the while loop. In your textbook, Common Errors 6.1 on page 232 shows a loop with exactly this problem.

The easiest solution to this problem is to use a step-by-step loop-building strategy. I'll show you one that I find useful in the online lecture titled How to Write Loops later this week. Your book also presents a similar strategy in the How To Section 6.1 on page 250. If you start by first writing the "mechanics" of the loop--bounds, precondition, and update--you are less likely to get distracted by the "real" statements and forget to add the update expression at the end of the loop.



Slide 12

Common Loop Errors

Duration: 00:01:12
Advance mode: Auto

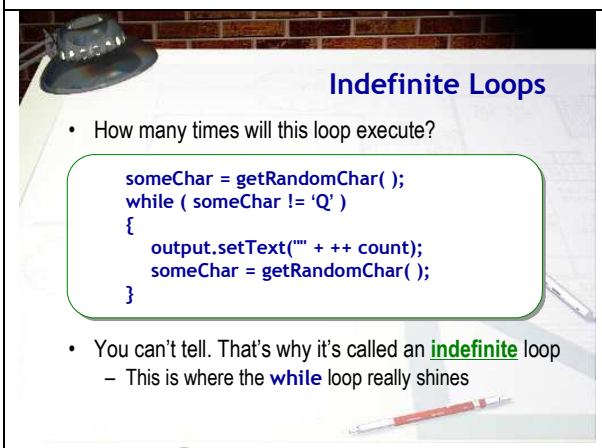
Notes:

This last problem is also not restricted to while loops; it affects selection statements and for loops as well.

If you put a semicolon after the condition of a while (or for) loop, the body of the loop becomes the null statement. Here's an example. Because the semicolon is placed immediately after the loop condition, the actual body of the loop is the null statement, not the code inside the braces. What you are saying with this code is: as long as cups is less than ten, do nothing.

This creates two problems:

1. If cups was originally less than ten, you will have an endless loop. Because nothing in the null body changes the value of cups, it will remain at its initial value.
2. If cups was greater-than or equal-to ten, you will not have an endless loop, but the statements that were intended to be executed as part of the loop body will be executed exactly once, not multiple times as you originally intended.



Slide 13

Indefinite Loops

Duration: 00:01:27
Advance mode: Auto

Notes:

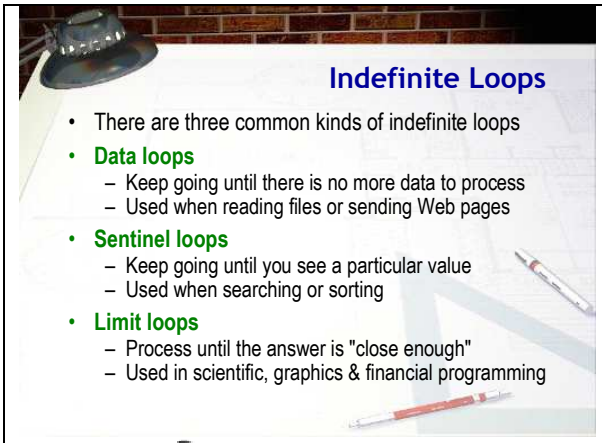
Although the for loop is the undisputed champ in the counted-loop arena, the while loop really comes into its own when the loop test condition is a little less clear-cut.

Let's look at an example. Here's a condensed fragment of code from an applet named Indefinite.java. The Indefinite applet uses a simple "event-testing" loop similar to the one shown here. Take a look at this loop, and see if you can guess how many times it will execute.

In the Indefinite applet, the character variable, someChar, is assigned a value between 0 and 127, returned by the method getRandomChar(). In the loop that follows, randomChar is tested, and, if it is not equal to the character 'Q', then the loop body is entered, where the number of repetitions is incremented and displayed.

Unless you have some kind of psychic powers, you can't know how many times this loop will run. That's why this kind of loop is called an indefinite loop. You can't tell by reading the source code how many repetitions it will make.

Even though this is not a counted loop, the body of the loop must still contain code that advances the loop's bound. If it does not, then, once entered, the loop will repeat forever. In this example, we advance toward the loop's bound by calculating a new random character for someChar.



Slide 14

Indefinite Loops

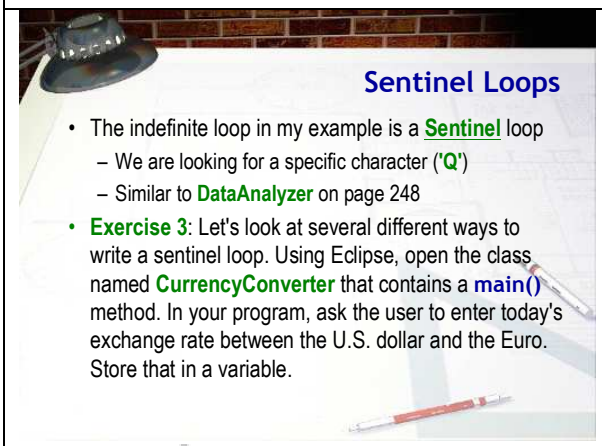
Duration: 00:01:23
Advance mode: Auto

Notes:

There are three commonly encountered types of indefinite loops that you should be aware of. Each of these indefinite loops uses a different sort of bounds:

- The first of these is called a data loop : A data loop is a loop that keeps going until there is no more data. This is used when reading files or sending Web pages over the Net.
- The second indefinite loop is known as a sentinel loop : A sentinel loop is an indefinite loop that looks for the presence of a special marker contained within its input. In the "generate random characters until you encounter Q" example, the Q is the sentinel. Sentinel loops are often used in searching, but have other uses as well.
- Finally we have the indefinite loops knows as limit loops. Limit loops are loops that end when another repetition of the loop won't get you any closer to your goal. Limit loops are often used in scientific calculations and other numeric algorithms, when stating a precise termination condition is not possible. Often this involves monitoring the difference between two variables, and stopping the loop when the difference passes a predetermined threshold.

Let's look first at sentinel loops.



Slide 15

Sentinel Loops

Duration: 00:01:30
Advance mode: Auto

Notes:

This particular indefinite loop in my example is a sentinel loop. A sentinel loop is a loop that looks through its input for some special value to end the loop. The special value is called the sentinel. As mentioned earlier, sentinels are commonly used in two situations:

- When searching through text to find a character or word.
- When used as an "out-of-bounds" value that marks the end of input.

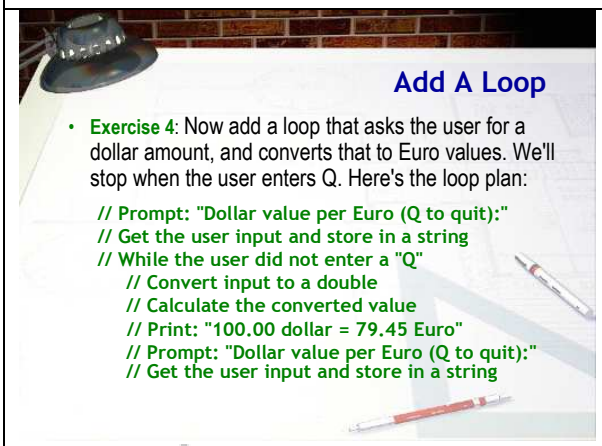
In this example, the sentinel is the letter 'Q', generated by the random number generator, and it is used to mark the end of valid input. This is very similar to the DataAnalyzer program on page 248 in your textbook.

To examine the different ways that we could write a sentinel loop, I want to complete Exercise 6.1 at the end of Chapter 6 in your textbook. I've already written a skeleton file, CurrencyConverter.java, and added it to the ic14 project. Go ahead and open the program and locate the main() method.

Inside the main() method, ask the user to enter today's exchange rate between the U.S. dollar and the Euro, and then store that in a variable. Express the rate as the number of Euro's per dollar (something like .78), not the number of Dollars per Euro.

Once you've done that, shoot me a screen-shot showing your

code.



Slide 16

Add A Loop

Duration: 00:02:23

Advance mode: Auto

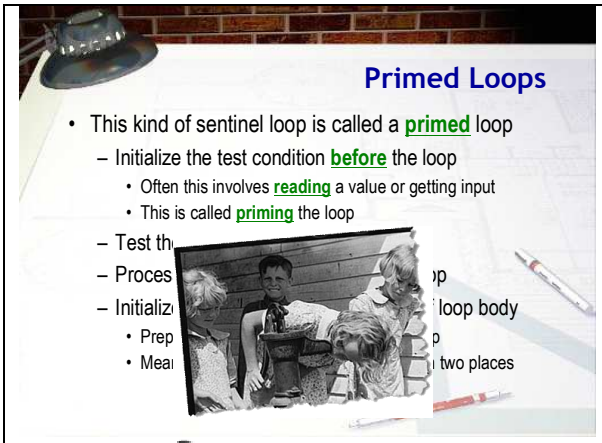
Notes:

Now, go ahead and complete the code to calculate the Euro value for different dollar amounts. Here's the loop plan:

- First prompt the user to enter a dollar value per Euro. Make sure you tell them to enter Q to quit. You'll use System.out.print() to display the prompt.
- Next, use a Scanner to read the user's input (as a String) and store it into a variable.
- Now, write an indefinite loop skeleton. The loop condition is that the user's input is not equal to "Q". When you write this condition, you can compare against the first character of the user's input (using charAt() and ==) or you can use one of the String comparison methods, like equals() or equalsIgnoreCase().
- Inside the body of the while loop you have several steps you need to carry out. If you enter the loop, that means that the user didn't enter a Q. We'll just make the assumption that if they don't enter Q that they'll enter a String representing a valid floating point number. Later, we'll learn how to validate the user's input. If the user entered a number, you'll need to use the Double.parseDouble() method to convert it to a double and store it in a double variable.
- Once you have the value entered by the user as a double, use the conversion factor (from Exercise 3) to calculate the number of Euros that they can buy and then print the answer showing both the number of dollars and the equivalent number of Euros.

After you complete your calculation and print the output, you'll need to see if the user wants to enter another value, or if they want to quit. To do that, you'll ask the user to enter a Dollar value per Euro once again, just like you did at the beginning, and then get the user input for the next repetition of the loop. These two lines at the bottom of the loop body will be almost identical to the two lines appearing immediately before the loop starts.

To finish up, for Exercise 4, run the program and show me a screen-shot of the output. You can use Google to find the current value of Dollar to the Euro.



Slide 17

Primed Loops

Duration: 00:01:06
Advance mode: Auto

Notes:

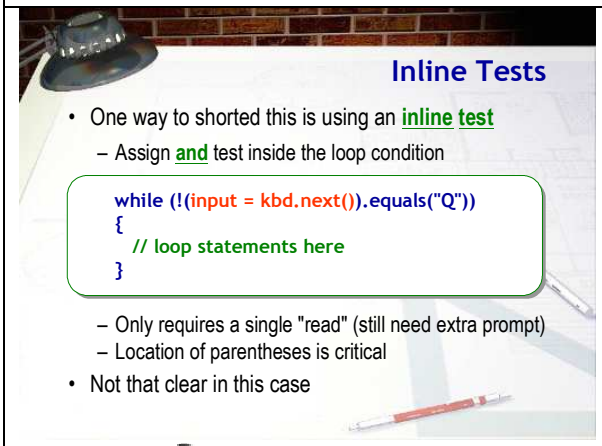
The kind of sentinel loop you just wrote for your CurrencyConverter program is called a primed loop because it works similar to an old-fashioned water pump like that in the picture here. With this hand-operated water pump, you always needed to keep a bucket of water next to the pump to get the pump suction going.

A primed loop is similar in that you have to initialize the test condition before the loop begins. Often that involves reading the first value from a series of input values before the loop even starts. This first value is known as the priming value.

Once the loop is primed, you:

- test the value against the sentinel in the loop condition
- process the variable as needed in the loop body
- *initialize the variable's value a second time at the end of the loop body, preparing for the next revolution of the loop.

This works just fine, but it means that you have almost the exact same code as that used to initialize your variable, in two places: before the loop and at the end of the loop.



Slide 18

Inline Tests

Duration: 00:01:32
Advance mode: Auto

Notes:

Because assignment is an operator in Java, and not a statement, there is a much shorter and more concise way to write the same kind of sentinel loop. This is known as an inline test. A loop employing an inline test is an extremely common Java idiom, and you should become familiar with it, even if you prefer the longer, more traditional primed loop.

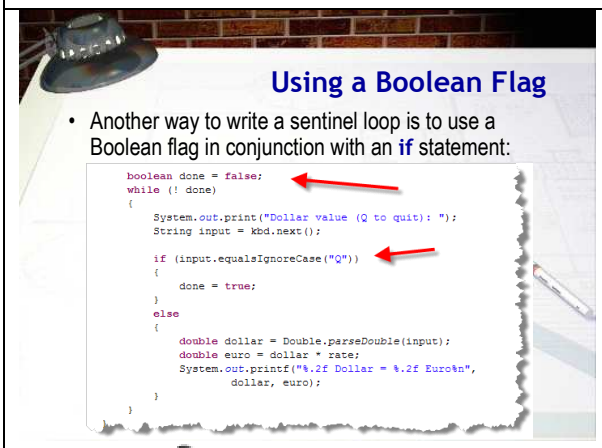
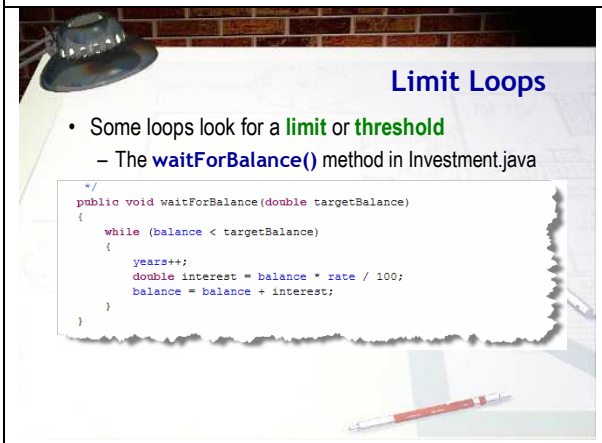
Using an inline test, the loop for your currency converter could be rewritten like this:

```
while (!( input = kbd.next().equals("Q"))
```

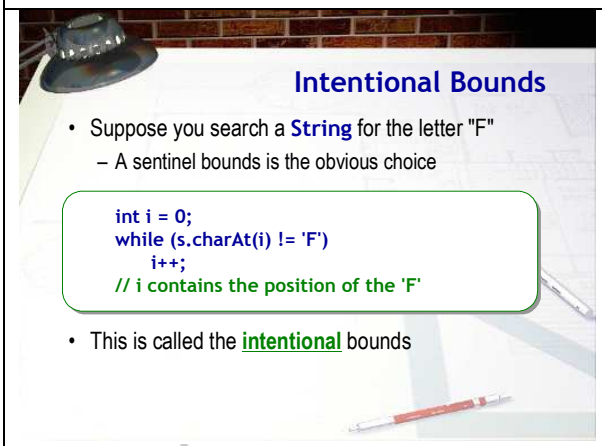
Now, you only have to initialize your input variable once--during the loop test--and not twice--before and at the end of the loop.

If you decide to use this style, pay special attention to the set of parentheses surrounding the assignment operation: (input = kbd.next())

The parentheses are required because the precedence of the dot operator used to call the equals() method is higher than the precedence of the assignment operator. If you omit the parentheses, Java will call the next() method and compare the word that it returns with "Q", generating a boolean value. When it comes time to assign the boolean value to the variable input, however, Java will choke because input is a String, and you cannot assign a boolean value to an String variable.

	Also, even though this is shorter, it's really not all that clear in this case.
 **Slide 19** **Using a Boolean Flag** Duration: 00:01:25 Advance mode: Auto	**Notes:** Another, more traditional way, to write a sentinel loop is to use a Boolean flag to control it. Because of this, sentinel loops are sometimes called "flag-controlled" loops. Here's a version of CurrrencyConverter that uses this style. Here are the points you should notice. • A boolean flag named done is initialized to false before the loop begins. This flag is then tested in the loop bounds. While the loop is not done, we'll continue. • We have only a single pair of input and read lines at the very beginning of the loop body (instead of one pair before and one pair after, like you have with a primed loop.) • Immediately after the input section, an if statement is used to determine whether we want to quit. If the input value is "Q" then the done flag is set to true. If it's not, then the else portion of the if statement executes and it calculates and prints the currency conversion value. This pattern is also sometimes called a "loop and a half" because we're actually using two tests: one to keep the loop going and one to decide how to set the Boolean flag. Section 6.4 in your book has much more information on processing sentinel values that you'll want to read.
 **Slide 20** **Limit Loops** Duration: 00:00:53	**Notes:** The limit loop is another kind of indefinite loop. In a limit loop, you don't look for a particular value as the bounds condition, like you would with a sentinel loop; instead you look for some limit or threshold to be exceeded. To see an example, open up the file Investment.java in Eclipse and locate the waitForBalance() method. As you can see, the method has one parameter, which is the target balance. Rather than using that as a sentinel, though, it's used as a limit. Each time through the loop the balance is increased by the amount of interest earned that year. Once the balance becomes larger than the targetBalance, then the loop exits. Next week, when we look at random numbers and simulations, you'll work with limit loops a little more.

Advance mode: Auto



Slide 21

Intentional Bounds

Duration: 00:01:18

Advance mode: Auto

Notes:

Before we finish up this lesson on the while loop, there are two more topics I want to cover briefly: the intentional and necessary bounds. These are not specific to while loops, however. You'll need to consider these in any loop you write.

Sentinel bounds are often used to search for a particular value in a set of data, such as looking for a particular word or character in a document. When used like this, however, sentinel-controlled loops have one great failing: they can't ensure that the value will actually be found.

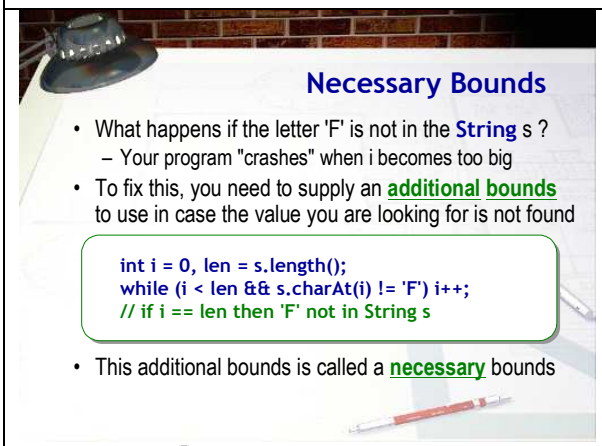
To handle this problem, you need to add an additional item to your loop-building toolkit--you must learn to differentiate between the intentional bounds and the necessary bounds.

Here's the difference.

Suppose you want to search the String s for the letter 'F', and return the position where the value is found. Using a simple sentinel loop you could write:

```
while (s.charAt(i) != 'F' ) i++;
```

This bounds is your intentional bounds; it is the sentinel value that will end your loop if all goes well.



Slide 22

Necessary Bounds

Duration: 00:01:34

Advance mode: Auto

Notes:

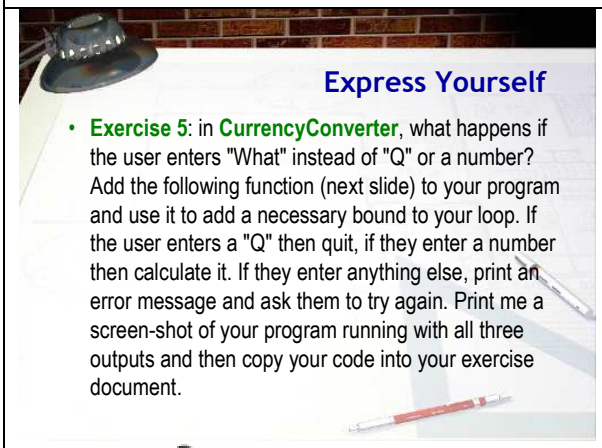
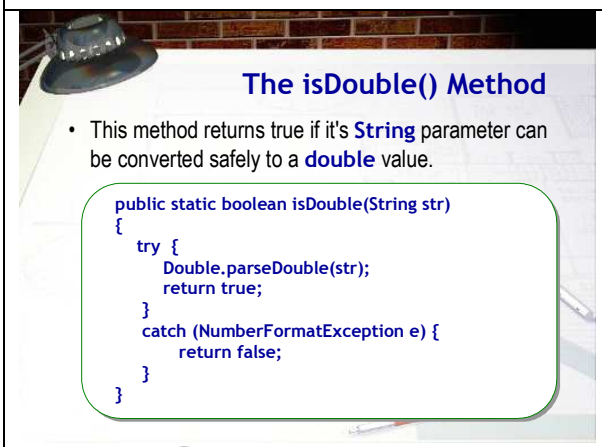
You can't just assume that all will go well, however. It is possible that:

- The letter 'F' might not occur in the String s
- The String s may have no characters

To handle these contingencies, you need to supply and additional bounds condition in the event that the value you're looking for is not found. Writing these bounds often involves crafting a compound Boolean condition using the logical operators, like this:

```
while ( i < len && s.charAt(i) != 'F') i++;
```

For instance, after this loop terminates, you can determine whether the character 'F' was found by testing the value of i. If i is equal to the len then either the String was empty (because both len and i would be 0) or because 'F' was not found after examining all of the characters in the String. If i is less than

	len, then you know that the character 'F' was found at position i. At other times, you may need to check for these bounds inside the loop body, using an if statement and a flag. The bounds used to handle these kinds of conditions are called the necessary bounds. Adding necessary bounds to a loop condition means that the loop can end for two (or more) reasons, not just one. When you write the loop postcondition statements, you have to take this into account.
 **Slide 23** **Express Yourself** Duration: 00:01:24 Advance mode: Auto	## Notes: OK, it's your turn one last time. In the CurrencyConverter program, it's possible that someone could enter something other than a number of a 'Q', just like it's possible that the letter 'F' wouldn't be found in the String. That means that we need to add another bounds condition, a necessary bounds, to the CurrencyConverter loop. I want you to rewrite the program so that: • If the user enters a 'Q' the loop quits • If the user enters a valid numeric amount, the output is calculated • If the user enters a non-numeric string amount, other than 'Q', then an error message is printed and the user is allowed to re-enter their input. Probably the easiest way to do this is to use the Boolean flag, "loop-and-a-half" technique shown on previous slides and covered in Section 6.4 in your textbook. To see if the String entered by the user is a number, add the predicate function isDouble() shown on the following slide to your program. When you're done, run your program and print me a screen-shot showing all three possible inputs: valid number, invalid String input, and the Q to quit. Then, copy and paste your code into your exercise document right below the Exercise 5 screen-shot.
 **Slide 24**	## Notes: To complete your exercise, just type this code into your CurrencyConverter program and then use the isDouble() method to check if the value entered by the user is actually a number. If it is not a number, and, it is not the letter "Q", then you need to tell the user that their input is invalid, and give them another chance.

The isDouble() Method Duration: 00:00:24 Advance mode: Auto	

